

System Architecture

Q: Give a high-level overview of the HealthApp system.

A: HealthApp is a multi-user collaborative medical Q&A system built with a Flask backend, SQL Server database, local LLM via Ollama, and a ChromaDB vector store. When a user asks a question, the system first checks if a semantically similar question was already answered (using cosine similarity on embeddings). If yes, it returns the cached answer instantly. If no, it retrieves relevant context from uploaded PDFs using RAG (Retrieval-Augmented Generation), sends it to a local LLM (LLaMA2), and returns the generated answer. All Q&A is stored in SQL Server and visible to all users in real time.

Q: What is RAG and why did you use it instead of fine-tuning the LLM?

A: RAG stands for Retrieval-Augmented Generation. Instead of retraining or fine-tuning the model on medical documents (which is expensive and slow), RAG retrieves relevant chunks from a vector store at query time and injects them into the LLM prompt as context. This means the model can answer questions about documents it was never trained on, and the knowledge base can be updated simply by uploading new PDFs — no retraining required. It is also more transparent because you can cite the exact source document and page number.

Embeddings & Cosine Similarity

Q: What is an embedding?

A: An embedding is a numerical representation of text as a vector of floating-point numbers (e.g., 768 numbers). Words or sentences with similar meanings end up with vectors that point in similar directions in high-dimensional space. In this project, we use the nomic-embed-text model via Ollama to convert both questions and PDF chunks into embeddings. These vectors are what allow the system to compare meaning rather than just matching keywords.

Q: What is cosine similarity and why use it instead of Euclidean distance?

A: Cosine similarity measures the angle between two vectors, not their magnitude. The formula is:

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

It returns a value between -1 and 1, where 1 means identical direction (very similar meaning) and 0 means unrelated. We use it instead of Euclidean distance because embeddings can vary in magnitude — a longer sentence produces a bigger vector, but that doesn't make it more semantically relevant. Cosine similarity normalizes for this, making it more reliable for semantic search.

Q: What is the 75% similarity threshold used for in this project?

A: When a new question comes in, the system computes its embedding and compares it against all existing question embeddings in the database using cosine similarity. If the similarity score is ≥ 0.75 (75%), the system treats the new question as effectively the same as the existing one and returns the

cached answer. This avoids redundant LLM calls and makes the system faster for repeated or rephrased questions. The threshold is configurable in config.py.

Q: Where are the question embeddings stored and how are they retrieved?

A: Question embeddings are stored as binary blobs (VARBINARY(MAX)) in the SQL Server questions table. When stored, the NumPy float64 array is serialized with .tobytes(). When retrieved, it is deserialized with np.frombuffer(emb_bytes, dtype=np.float64). The system then loops through all questions in the current chat and computes cosine similarity against the new question's embedding to find a cache hit.

Q: Why are PDF chunk embeddings stored in ChromaDB while question embeddings are stored in SQL Server?

A: They serve different purposes. ChromaDB is a purpose-built vector database optimized for semantic similarity search over large numbers of document chunks — it uses approximate nearest neighbor (ANN) algorithms internally for speed. SQL Server stores question embeddings because there are far fewer questions than document chunks, and it keeps all relational data (users, chats, answers) in one place. Using ChromaDB for questions would add complexity without benefit at this scale.

 PDF Processing & ChromaDB

Q: Walk through what happens when a user uploads a PDF.

A: The upload is handled asynchronously in a background thread so the UI stays responsive. The flow is:

1. The PDF is saved to the uploads/ folder
 2. pypdf reads and extracts text from each page
 3. Pages with no text are skipped
 4. The text is split into 500-character chunks with 50-character overlap using RecursiveCharacterTextSplitter
 5. Each chunk is embedded using nomic-embed-text in batches of 50
 6. The chunks and their embeddings are added to ChromaDB
 7. Document metadata (filename, uploader, date) is saved to SQL Server
-

Q: Why split PDFs into chunks? Why not embed the whole page?

A: LLM context windows have token limits, and embedding an entire page would produce a single coarse vector that blends many topics together, reducing retrieval precision. Smaller chunks (500 characters) mean each embedding represents a focused concept, so when a user asks a specific

question, the retrieved chunks are more likely to contain the directly relevant passage. The 50-character overlap ensures no information is cut off at a chunk boundary.

Q: How does the system know which page number to cite in the answer?

A: During PDF processing, each chunk's metadata includes the actual 1-indexed page number (`page_num + 1`). When the LLM generates an answer, the system looks at the metadata of the top-ranked retrieved chunk (`docs[0].metadata["page"]`) and returns that as `source_page`. This is why we store `page_num + 1` not `page_num` — so the cited page matches what the user sees when they open the PDF.

Q: What is `similarity_search_with_score` and what does ChromaDB actually return?

A: `similarity_search_with_score` retrieves the top-K most relevant document chunks for a query and returns (document, distance) pairs. Importantly, ChromaDB returns **distance** (lower = more similar), not similarity. The code converts distance to similarity using:

```
python
```

```
similarity = 1.0 / (1.0 + distance)
```

```
...
```

This maps distance 0 → similarity 1.0 (identical), and larger distances → smaller similarities approaching 0.

```
---
```

🤖 LLM & Answer Generation

****Q:** How is the answer generated from retrieved documents?**

****A:**** After retrieving the top-5 most relevant chunks from ChromaDB, the system builds a context string by concatenating the chunks labeled `[Document 1]`, `[Document 2]`, etc. This context is injected into a prompt template:

```
...
```

Use the context below to answer the question. Be concise and specific.

If you don't know the answer, say "I don't have enough information."

Context: {context}

Question: {question}

Answer (be concise):

This prompt is sent directly to the LLaMA2 LLM via `llm.invoke(prompt_text)` and the response text is returned.

Q: How is confidence calculated?

A: Confidence is calculated from the cosine similarity scores of the retrieved chunks, not from the LLM itself. The steps are:

1. Retrieve similarity scores for the top-K chunks
2. If max score < 0.3, return low confidence (0.2)
3. Otherwise compute an exponentially weighted average of scores (higher scores weighted more)
4. Apply a consistency penalty for high variance across scores ($1 - \text{std} * 0.5$)
5. Then apply quality adjustments: reduce if answer is very short (<20 chars), reduce if LLM says "I don't know", boost if answer is long and top similarity is >0.8

This gives a confidence score between 0 and 1.

Q: What is `get_cached_answer_confidence` and why adjust confidence for cached answers?

A: When a cached answer is returned, the confidence of the original answer may not perfectly reflect the new question, because the new question might be only 76% similar rather than 95% similar. `get_cached_answer_confidence` adjusts the original confidence by a factor based on how similar the new question is to the cached one:

- Similarity ≥ 0.90 → keep original confidence
 - Similarity 0.75–0.90 → scale down linearly (factor 0.7 to 1.0)
 - Similarity < 0.75 → halve the confidence
-

Q: Why use a local LLM (Ollama/LLaMA2) instead of an API like OpenAI?

A: In a healthcare context, patient data and medical documents are highly sensitive. Using a local LLM means no data ever leaves the local network — there is no risk of sending medical information to third-party APIs. It also eliminates per-token API costs, removes dependency on internet connectivity, and avoids rate limits. The tradeoff is lower answer quality compared to GPT-4, but for domain-specific RAG over controlled documents, a local model is often sufficient.

Q: Describe the database schema and how the tables relate.

A: There are 5 tables with a clear hierarchy:

- users — stores username and hashed password (parent of everything)
- chat_sections — each chat belongs to a user (FK to users)
- questions — each question belongs to a chat and a user (FK to both)
- answers — each answer belongs to exactly one question (FK to questions), stores the answer text, confidence, source doc, source page, and feedback
- documents — metadata about uploaded PDFs, belongs to a user

The relationship chain is: users → chat_sections → questions → answers and users → documents.

Q: Why store passwords as SHA-256 hashes instead of plain text?

A: If the database is ever compromised, plain text passwords expose all users. SHA-256 hashing means only the hash is stored. When a user logs in, the entered password is hashed and compared to the stored hash — the original password is never stored or transmitted. In production you would use bcrypt or Argon2 instead of SHA-256 because those are specifically designed to be slow, making brute-force attacks harder.

Q: Why does the questions table use VARBINARY(MAX) for embeddings?

A: The nomic-embed-text model produces embeddings as NumPy float64 arrays. Serialized with .tobytes(), each embedding is a sequence of raw bytes. VARBINARY(MAX) can store up to 2GB of binary data, more than enough for any embedding. The alternative would be storing comma-separated floats as text (NVARCHAR(MAX)), but that is slower to insert, retrieve, and deserialize.

Q: Why use OUTPUT INSERTED.answer_id in the INSERT statement?

A: SQL Server's OUTPUT clause returns values from rows affected by an INSERT, UPDATE, or DELETE in a single round-trip. Without it, you would need a separate SELECT @@IDENTITY or SELECT SCOPE_IDENTITY() query. OUTPUT INSERTED.answer_id is safer because @@IDENTITY can return the wrong value in triggers, and SCOPE_IDENTITY() requires an extra query. This is also how the answer_id is returned to the frontend for the feedback system.

Q: What was the answer_id = 0 bug and why did it break the frontend?

A: When DBCC CHECKIDENT ('answers', RESEED, 0) is run to reset the identity column, the **next** inserted row gets ID = 0, not 1. In JavaScript, the number 0 is **falsy** — meaning if (answer_id) evaluates to false when answer_id = 0. This caused the feedback buttons to never render. The fix was twofold: reseed with the correct value so IDs start from 1, and update the JavaScript check from if (answer_id) to if (answer_id !== null && answer_id !== undefined).

Flask Backend

Q: How does session management work in Flask?

A: Flask uses server-side sessions backed by a signed cookie. When a user logs in, `session['user_id']` and `session['username']` are set. Flask signs the cookie with `app.secret_key` so the client cannot tamper with it. On every subsequent request, Flask reads the cookie and verifies the signature, making the session data available. All protected routes check if `'user_id'` not in session and return 401 if not authenticated.

Q: Why is PDF processing done in a background thread?

A: PDF processing can take 10–60 seconds for large documents — embedding hundreds of chunks through Ollama is slow. If this ran synchronously in the Flask request handler, the HTTP response would time out and the UI would freeze. Instead, the task is put into a Queue and a daemon thread (`background_processor`) picks it up. The frontend polls `/upload_status/<task_id>` every 500ms and shows a progress bar based on the callback updates.

Q: How does the polling mechanism work for batch processing?

A: When an Excel file is uploaded, the questions list is put into `batch_queue` and a `batch_processor` thread processes them one by one, updating `batch_status[task_id]` with progress. The frontend calls `/batch_status/<task_id>` every second and updates the progress bar. When status becomes "completed", a download button appears. This is a simple long-polling approach — a production system might use WebSockets or Server-Sent Events for real-time push updates.

Frontend

Q: How does the real-time collaborative feature work?

A: The frontend polls the `/history?chat_id=X` endpoint every 3 seconds using `setInterval`. When the response comes back with more messages than the last count (`history.length !== lastMessageCount`), the messages div is re-rendered. This means if User A and User B are in the same chat, User B will see User A's question and answer within 3 seconds without refreshing. Polling pauses when the browser tab is hidden (using the `visibilitychange` event) to save resources.

Q: How does the feedback system work end to end?

A: When an answer is rendered, its `answer_id` from the database is embedded in the button's `onclick` attribute. When the user clicks "✓ Correct", `markCorrect(answer_id)` sends a POST to `/feedback` with `{answer_id, is_correct: true}`. Flask calls `mark_answer_feedback(answer_id, True)` which runs `UPDATE answers SET is_correct = 1 WHERE answer_id = ?`. If "X Incorrect" is clicked, a textarea appears, the user types a correction, and `submitCorrection(answer_id)` sends the corrected text, which updates both `is_correct = 0` and `answer_text` in the database.

Q: What is the similarity threshold feature card referring to when it says "75% threshold"?

A: It refers to the cosine similarity cache threshold. If a new question has $\geq 75\%$ cosine similarity to an already-answered question in the same chat, the system returns the existing answer instead of calling the LLM again. This is the `SIMILARITY_THRESHOLD = 0.75` value in `config.py`. It is a balance: too high (e.g., 0.95) and nearly identical questions still trigger redundant LLM calls; too low (e.g., 0.5) and different questions incorrectly share answers.

Q: How does the export feature work?

A: The export button calls `exportToExcel()` in JavaScript. A `confirm()` dialog asks whether to export the current chat or all chats, then navigates to `/export?chat_id=X` or `/export`. Flask calls `get_all_qa_for_export()` which runs a JOIN across questions, answers, and chat_sections. The results are converted to a list of dictionaries, put into a Pandas DataFrame, written to an in-memory BytesIO buffer using `openpyxl`, and sent as a file download with `send_file()`. The buffer never touches the disk.

Q: What is the difference between `is_from_history` and the similarity cache?

A: `is_from_history` is a database flag on an answer row that records whether that answer was *served from cache* at the time it was given. The similarity cache is the *mechanism* that checks for similar questions at runtime. So if Question B was answered by returning the cached answer from Question A, Question B's answer row has `is_from_history = True`. This lets you analyze later what percentage of questions were served from cache versus requiring a new LLM call.